Slash (id = 62) is a member of both Guns N' Roses (id = 13) and Velvet Revolver (id = 54). Singer Scott Weiland (id = 81) is a member of both Stone Temple Pilots (id = 27) and Velvet Revolver (still id = 54). The `Band_Musicians` table is how the band memberships get expressed. There is a row for Slash and Guns N' Roses (id = 36), a row for Slash and Velvet Revolver (id = 63), a row for Scott and STP (id = 45), and a row for Scott and Velvet Revolver (id = 64).

As a many-to-many relationship is two-way, you can put the `ManyToManyField` on either of the two participating objects and get a similar result. Semantically though, it tends to be clearer if the object representing a conglomerate is where the relationship gets placed. A band is a group of people, so it feels clearer if the `ManyToManyField` is on the `Band` object. Besides the relationship, the `Band` model also needs a name field. Open RiffMates/bands/models.py, and add the code for the `Band` object shown in the following listing.

Listing 4.14 `Venue` and `Room` models and their one-to-many relationship

```
# RiffMates/bands/models.py
...
class Band(models.Model):
    name = models.CharField(max_length=20)
    musicians = models.ManyToManyField(Musician)

    def __str__(self):
        return f"Band(id={self.id}, name={self.name})"
```

As with earlier changes to Django `Model` classes, you need to run the `makemigrations` and `migrate` management commands to sync the database with your `Model` code:

```
(venv) RiffMates$ python manage.py makemigrations bands
Migrations for 'bands':
  bands/migrations/0003_band.py
    - Create model Band
(venv) RiffMates$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bands, contenttypes, sessions
Running migrations:
  Applying bands.0003_band... OK
```

`ForeignKey` fields get populated as an attribute when you create an object, like you did passing a `Venue` into your `Room` object. In a many-to-many relationship, the relation is in a separate table, meaning the relation needs to be created as a separate step. Before doing anything else, you need a `Band` object; enter the Django REPL, and create it as follows:

```
>>> from bands.models import Band
>>> beatles = Band.objects.create(name="The Beatles")
```

To add Mr. Lennon to The Beatles, you need a reference. You can query him based on his ID if you remember it, or as his last name is currently unique, you can run a `.get()` query based on that: